

2303A51602

Batch:25

Task 1: AI-Assisted Syntax and Code Quality Review

Scenario

You join a development team and are asked to review a junior developer's Python script that fails to run correctly due to basic coding mistakes. Before deployment, the code must be corrected and standardized.

Task Description

You are given a Python script containing:

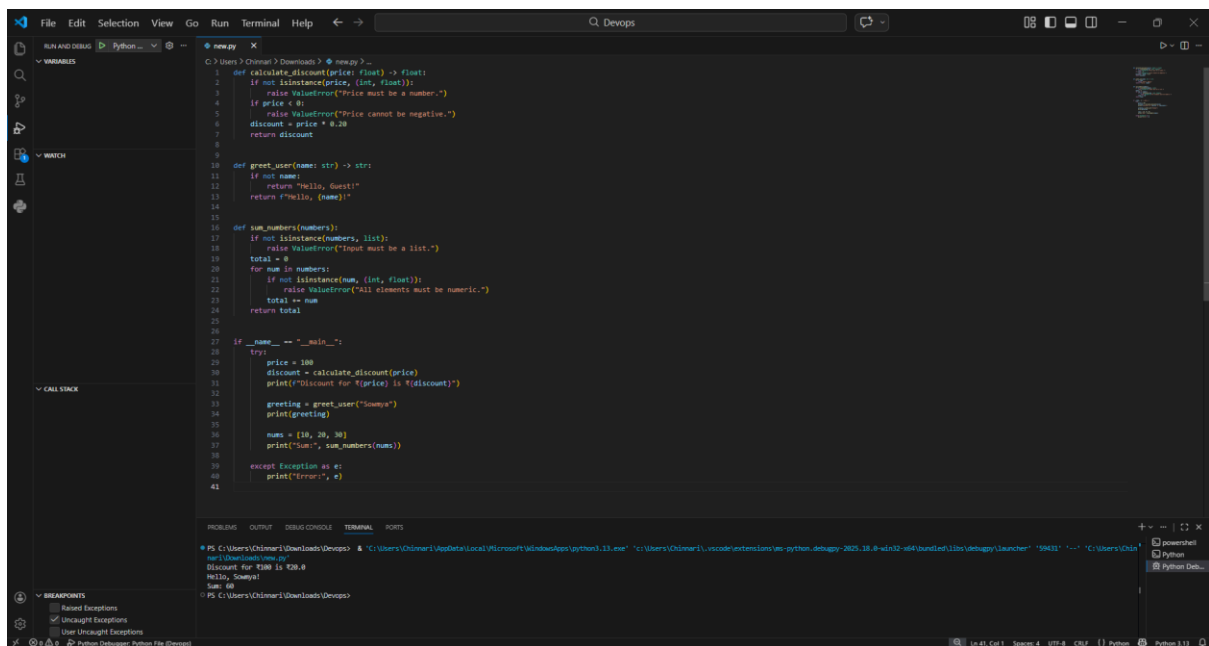
- Syntax errors
- Indentation issues
- Incorrect variable names
- Faulty function calls

Use an AI tool (GitHub Copilot / Cursor AI) to:

- Identify all syntactic and structural errors
- Correct them systematically
- Generate an explanation of each fix made

Expected Outcome

- Fully corrected and executable Python code
- AI-generated explanation describing:
 - o Syntax fixes
 - o Naming corrections
 - o Structural improvements
- Clean, readable version of the script



Task 2: Performance-Oriented Code Review

Scenario

A data processing function works correctly but is inefficient and slows down the system when large datasets are used.

Task Description

You are provided with a function that identifies duplicate values in a list using inefficient nested loops.

Using AI-assisted code review:

- Analyze the logic for performance bottlenecks
- Refactor the code for better time complexity
- Preserve the correctness of the output

Ask the AI to explain:

- Why the original approach was inefficient
- How the optimized version improves performance

Expected Outcome

- Optimized duplicate-detection logic (e.g., using sets or hash-based structures)
- Improved time complexity
- AI explanation of performance improvement
- Clean, readable implementation

```
def find_duplicates_slow(data):
    duplicates = []
    for i in range(len(data)):
        for j in range(i+1, len(data)):
            if data[i] == data[j] and data[i] not in duplicates:
                duplicates.append(data[i])
    return duplicates

def find_duplicates_fast(data):
    seen = set()
    duplicates = set()
    for item in data:
        if item in seen:
            duplicates.add(item)
        else:
            seen.add(item)
    return list(duplicates)

def explanation():
    print("AI Code Review Explanation:\n")
    print("Why the original approach was inefficient:")
    print("1. It used nested loops, resulting in O(n^2) time complexity.")
    print("2. Each element was compared with every other element.")
    print("3. Checking duplicates in a list also takes extra time.")
    print("How the optimized version improves performance:")
    print("1. Uses a set (hash-based structure) for constant-time lookup O(1).")
    print("2. Traverses the list only once, giving O(n) time complexity.")
    print("3. Significantly faster for large datasets.")

if __name__ == "__main__":
    data = [1, 2, 3, 4, 1, 5, 6, 3, 7, 1]
    print("Duplicates (Slow Method):", find_duplicates_slow(data))
    print("Duplicates (Optimized Method):", find_duplicates_fast(data))
    print()
    explanation()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\Chinar\Downloads\Devs> cd "C:\Users\Chinar\Downloads\Devs" & cd "C:\Users\Chinar\AppData\Local\Microsoft\WindowsApps\python3.11.exe" "C:\Users\Chinar\Downloads\Devs\python-detuggy-2025.10.8-win32-x64\python3.11.exe" "C:\Users\Chinar\Downloads\Devs\new.py"

Duplicates (Slow Method): [1, 2, 3]

Duplicates (Optimized Method): [1, 2, 3]

AI Code Review Explanation:

Why the original approach was inefficient:

1. It used nested loops, resulting in O(n^2) time complexity.
2. Each element was compared with every other element.
3. Checking duplicates in a list also takes extra time.

How the optimized version improves performance:

1. Uses a set (hash-based structure) for constant-time lookup O(1).
2. Traverses the list only once, giving O(n) time complexity.
3. Significantly faster for large datasets.

```
def find_duplicates_fast(data):
    duplicates = set()
    for item in data:
        if item in seen:
            duplicates.add(item)
        else:
            seen.add(item)
    return list(duplicates)

def explanation():
    print("AI Code Review Explanation:\n")
    print("Why the original approach was inefficient:")
    print("1. It used nested loops, resulting in O(n^2) time complexity.")
    print("2. Each element was compared with every other element.")
    print("3. Checking duplicates in a list also takes extra time.")
    print("How the optimized version improves performance:")
    print("1. Uses a set (hash-based structure) for constant-time lookup O(1).")
    print("2. Traverses the list only once, giving O(n) time complexity.")
    print("3. Significantly faster for large datasets.")

if __name__ == "__main__":
    data = [1, 2, 3, 4, 1, 5, 6, 3, 7, 1]
    print("Duplicates (Slow Method):", find_duplicates_slow(data))
    print("Duplicates (Optimized Method):", find_duplicates_fast(data))
    print()
    explanation()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\Chinar\Downloads\Devs> cd "C:\Users\Chinar\Downloads\Devs" & cd "C:\Users\Chinar\AppData\Local\Microsoft\WindowsApps\python3.11.exe" "C:\Users\Chinar\Downloads\Devs\python-detuggy-2025.10.8-win32-x64\python3.11.exe" "C:\Users\Chinar\Downloads\Devs\new.py"

Duplicates (Slow Method): [1, 2, 3]

Duplicates (Optimized Method): [1, 2, 3]

AI Code Review Explanation:

Why the original approach was inefficient:

1. It used nested loops, resulting in O(n^2) time complexity.
2. Each element was compared with every other element.
3. Checking duplicates in a list also takes extra time.

How the optimized version improves performance:

1. Uses a set (hash-based structure) for constant-time lookup O(1).
2. Traverses the list only once, giving O(n) time complexity.
3. Significantly faster for large datasets.

Task 3: Readability and Maintainability Refactoring

Scenario

A working script exists in a project, but it is difficult to understand due to poor naming, formatting, and structure. The team wants it rewritten for long-term maintainability.

Task Description

You are given a poorly structured Python function with:

- Cryptic function names
- Poor indentation

- Unclear variable naming
- No documentation

Use AI-assisted review to:

- Refactor the code for clarity
- Apply PEP 8 formatting standards
- Improve naming conventions
- Add meaningful documentation

Expected Outcome

- Clean, well-structured code
- Descriptive function and variable names
- Proper indentation and formatting
- Docstrings explaining the function purpose
- AI explanation of readability improvements

```

1 def calculate_student_average(marks):
2     if not isinstance(marks, list) or len(marks) == 0:
3         raise ValueError("Marks must be a non-empty list.")
4
5     total_marks = 0
6     for score in marks:
7         if not isinstance(score, (int, float)):
8             raise ValueError("All marks must be numbers.")
9         total_marks += score
10
11     average = total_marks / len(marks)
12     return average
13
14
15 def ai_explanation():
16     print("AI Refactoring Explanation:\n")
17
18     print("Readability Improvements:")
19     print("1. Renamed function to 'calculate_student_average' for clarity.")
20     print("2. Replaced unclear variable names with descriptive names like 'marks', 'score', and 'total_marks'.")
21     print("3. Applied proper indentation and spacing following PEP 8 standards.")
22
23     print("Maintainability Improvements:")
24     print("1. Added a clear docstring explaining purpose, arguments, and return value.")
25     print("2. Added input validation and error handling.")
26     print("3. Structured the code into a reusable function.")
27
28     print("Overall Result:")
29     print("The code is now easier to read, understand, modify, and maintain.")
30
31
32 if __name__ == "__main__":
33     student_marks = [85, 90, 78, 92, 88]
34
35     try:
36         avg = calculate_student_average(student_marks)
37         print(f"Average Marks: {avg}")
38     except Exception as e:
39         print(f"Error: {e}")

```

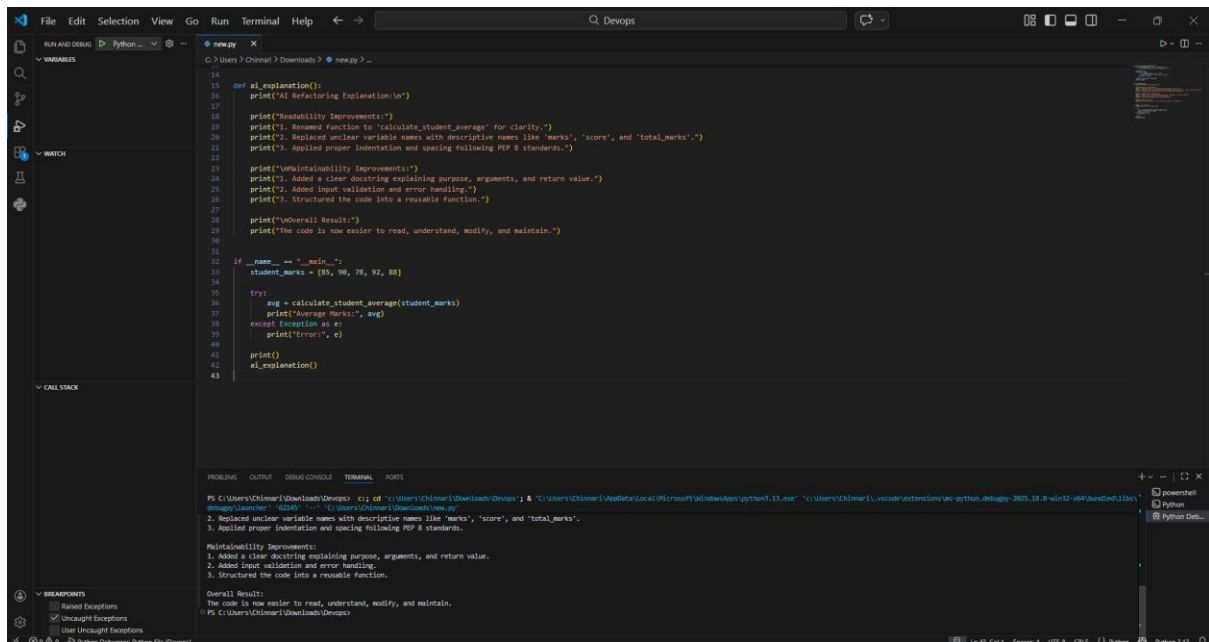
AI Refactoring Explanation:

Readability Improvements:

1. Renamed function to 'calculate_student_average' for clarity.
2. Replaced unclear variable names with descriptive names like 'marks', 'score', and 'total_marks'.
3. Applied proper indentation and spacing following PEP 8 standards.

Maintainability Improvements:

1. Added a clear docstring explaining purpose, arguments, and return value.
2. Added input validation and error handling.
3. Structured the code into a reusable function.



Task 4: Secure Coding and Reliability Review

Scenario

A backend function retrieves user data from a database but has security vulnerabilities and poor error handling, making it unsafe for production deployment.

Task Description

You are given a Python script that:

- Uses unsafe SQL query construction
- Has no input validation
- Lacks exception handling

Use AI tools to:

- Identify security vulnerabilities
- Refactor the code using safe coding practices
- Add proper exception handling
- Improve robustness and reliability

Expected Outcome

- Secure SQL queries using parameterized statements
- Input validation logic
- Try-except blocks for runtime safety
- AI-generated explanation of security improvements

- Production-ready code structure give code for this remove comments

```

1 import sqlite3
2 def get_user_by_id(user_id):
3     if not isinstance(user_id, int) or user_id <= 0:
4         raise ValueError("Invalid user ID")
5
6     connection = None
7     try:
8         connection = sqlite3.connect("users.db")
9         cursor = connection.cursor()
10
11         query = "SELECT id, name, email FROM users WHERE id = ?"
12         cursor.execute(query, (user_id,))
13
14         result = cursor.fetchone()
15
16         if result:
17             return {"id": result[0], "name": result[1], "email": result[2]}
18         else:
19             return None
20
21 except sqlite3.Error as db_error:
22     return {"error": "Database error: (db_error)"}
23
24 except Exception as e:
25     return {"error": "Unexpected error: (e)"}
26
27 finally:
28     if connection:
29         connection.close()
30
31
32 def ai_explanation():
33     print("AI Security Review Explanation:")
34
35     print("Security Improvements:")
36     print("1. Replaced unsafe SQL string concatenation with parameterized query.")
37     print("2. Added input validation to ensure user_id is a positive integer.")
38     print("3. Prevented SQL injection vulnerabilities.")
39
40
41     print("Reliability Improvements:")
42     print("1. Added try-except blocks to handle database and runtime errors.")
43     print("2. Ensured database connection is always closed using finally block.")
44     print("3. Returned structured error messages for safer backend handling.")

```

AI Security Review Explanation:

Security Improvements:

1. Replaced unsafe SQL string concatenation with parameterized query.
2. Added input validation to ensure user_id is a positive integer.
3. Prevented SQL injection vulnerabilities.

Reliability Improvements:

1. Added try-except blocks to handle database and runtime errors.
2. Ensured database connection is always closed using finally block.
3. Returned structured error messages for safer backend handling.

Task 5: AI-Based Automated Code Review Report

Scenario

Your team uses AI tools to perform automated preliminary code reviews before human review, to improve code quality and consistency across projects.

Task Description

You are provided with a poorly written Python script.

Using AI-assisted review:

- Generate a structured code review report that evaluates:

- o Code readability
- o Naming conventions
- o Formatting and style consistency
- o Error handling
- o Documentation quality
- o Maintainability

The task is not just to fix the code, but to analyze and report on quality issues.

Expected Outcome

- AI-generated review report including:

- o Identified quality issues
- o Risk areas
- o Code smell detection
- o Improvement suggestions
- Optional improved version of the code
- Demonstration of AI as a code reviewer, not just a code Generator

The screenshot shows a VS Code editor with a Python file named `new.py` and its output in the terminal. The script defines two functions: `poorly_written_function` and `improved_function`. The `improved_function` includes input validation, error handling, and meaningful naming. The terminal output shows the execution of `ai_code_review_report()`, which generates a detailed report on code quality issues.

```

1 def poorly_written_function(a, b):
2     c = a / b
3     print("Result:", c)
4     return c
5
6
7 def improved_function(number1, number2):
8     if not isinstance(number1, (int, float)) or not isinstance(number2, (int, float)):
9         raise ValueError("Inputs must be numeric")
10
11     if number2 == 0:
12         raise ValueError("Division by zero is not allowed")
13
14     result = number1 / number2
15     return result
16
17
18 def ai_code_review_report():
19     print("AI Automated Code Review Report")
20
21     print("\n1. Code Readability Issues:")
22     print("- Function name was unclear and not descriptive.")
23     print("- Variable names (a, b, c) did not indicate purpose.")
24     print("- Logic and output were mixed together.")
25
26     print("\n2. Naming Convention Issues:")
27     print("- Did not follow descriptive naming standards.")
28     print("- Improved version uses meaningful names like number1 and result.")
29
30     print("\n3. Formatting and Style Consistency:")
31     print("- No consistent spacing or structure.")
32     print("- Improved version follows standard Python formatting.")
33
34     print("\n4. Error Handling Problems:")
35     print("- No validation for invalid inputs.")
36     print("- No protection against division by zero.")
37     print("- Improved version adds input validation and exceptions.")
38
39     print("\n5. Documentation Quality:")
40
41     print("\n6. Maintainability Risks:")
42     print("- Hard to extend due to poor naming and structure.")
43     print("- Mixing computation with printing reduces reusability.")
44
45     print("\n7. Code Smells Detected:")
46     print("- Magic variables.")
47     print("- Lack of validation.")
48     print("- Tight coupling between logic and UI output.")
49
50     print("\n8. Improvement Suggestions:")
51     print("- Use descriptive names.")
52     print("- Separate logic from presentation.")
53     print("- Add validation and error handling.")
54     print("- Follow consistent formatting standards.")
55
56     print("\nResult:")
57     print("Improved version is safer, more readable, and maintainable.")
58
59
60 if __name__ == "__main__":
61     try:
62         print("Improved Result:", improved_function(10, 2))
63     except Exception as e:
64         print("Error:", e)
65
66
67

```

AI Automated Code Review Report

- Code Readability Issues:**
 - Function name was unclear and not descriptive.
 - Variable names (a, b, c) did not indicate purpose.
 - Logic and output were mixed together.
- Naming Convention Issues:**
 - Did not follow descriptive naming standards.
 - Improved version uses meaningful names like number1 and result.
- Formatting and Style Consistency:**
 - No consistent spacing or structure.
 - Improved version follows standard Python formatting.
- Error Handling Problems:**
 - No validation for invalid inputs.
 - No protection against division by zero.
 - Improved version adds input validation and exceptions.
- Documentation Quality:**
- Maintainability Risks:**
 - Hard to extend due to poor naming and structure.
 - Mixing computation with printing reduces reusability.
- Code Smells Detected:**
 - Magic variables.
 - Lack of validation.
 - Tight coupling between logic and UI output.
- Improvement Suggestions:**
 - Use descriptive names.
 - Separate logic from presentation.
 - Add validation and error handling.
 - Follow consistent formatting standards.

Result:
Improved version is safer, more readable, and maintainable.

The screenshot shows the same VS Code editor with the `new.py` file, now containing the improved version of the code. The terminal output shows the execution of `ai_code_review_report()`, which generates a detailed report on code quality issues, including input validation, error handling, and meaningful naming.

```

18 def ai_code_review_report():
19     print("AI Automated Code Review Report")
20
21     print("\n1. Code Readability Issues:")
22     print("- Function name was unclear and not descriptive.")
23     print("- Variable names (a, b, c) did not indicate purpose.")
24     print("- Logic and output were mixed together.")
25
26     print("\n2. Naming Convention Issues:")
27     print("- Did not follow descriptive naming standards.")
28     print("- Improved version uses meaningful names like number1 and result.")
29
30     print("\n3. Formatting and Style Consistency:")
31     print("- No consistent spacing or structure.")
32     print("- Improved version follows standard Python formatting.")
33
34     print("\n4. Error Handling Problems:")
35     print("- No validation for invalid inputs.")
36     print("- No protection against division by zero.")
37     print("- Improved version adds input validation and exceptions.")
38
39     print("\n5. Documentation Quality:")
40
41     print("\n6. Maintainability Risks:")
42     print("- Hard to extend due to poor naming and structure.")
43     print("- Mixing computation with printing reduces reusability.")
44
45     print("\n7. Code Smells Detected:")
46     print("- Magic variables.")
47     print("- Lack of validation.")
48     print("- Tight coupling between logic and UI output.")
49
50     print("\n8. Improvement Suggestions:")
51     print("- Use descriptive names.")
52     print("- Separate logic from presentation.")
53     print("- Add validation and error handling.")
54     print("- Follow consistent formatting standards.")
55
56     print("\nResult:")
57     print("Improved version is safer, more readable, and maintainable.")
58
59
60 if __name__ == "__main__":
61     try:
62         print("Improved Result:", improved_function(10, 2))
63     except Exception as e:
64         print("Error:", e)
65
66
67

```

AI Automated Code Review Report

- Code Readability Issues:**
 - Function name was unclear and not descriptive.
 - Variable names (a, b, c) did not indicate purpose.
 - Logic and output were mixed together.
- Naming Convention Issues:**
 - Did not follow descriptive naming standards.
 - Improved version uses meaningful names like number1 and result.
- Formatting and Style Consistency:**
 - No consistent spacing or structure.
 - Improved version follows standard Python formatting.
- Error Handling Problems:**
 - No validation for invalid inputs.
 - No protection against division by zero.
 - Improved version adds input validation and exceptions.
- Documentation Quality:**
- Maintainability Risks:**
 - Hard to extend due to poor naming and structure.
 - Mixing computation with printing reduces reusability.
- Code Smells Detected:**
 - Magic variables.
 - Lack of validation.
 - Tight coupling between logic and UI output.
- Improvement Suggestions:**
 - Use descriptive names.
 - Separate logic from presentation.
 - Add validation and error handling.
 - Follow consistent formatting standards.

Result:
Improved version is safer, more readable, and maintainable.

